

Inter-Integrated Circuit (I2C) Module

This chapter describes the inter-integrated circuit (I2C or I²C) module. The I2C is a multi-master communication module providing an interface between the Texas Instruments (TI) microcontroller and devices compliant with Philips Semiconductor I²C-bus specification version 2.1 and connected by an I2C-bus. This module will support any slave or master I2C compatible device.

Topic	Page
30.1 Overview.....	1656
30.2 I2C Module Operation.....	1660
30.3 I2C Operation Modes.....	1664
30.4 I2C Module Integrity	1666
30.5 Operational Information.....	1668
30.6 I2C Control Registers	1671
30.7 Sample Waveforms	1692

30.1 Overview

The I2C has the following features:

- Compliance to the Philips I²C bus specification, v2.1 (*The I2C Specification*, Philips document number 9398 393 40011)
 - Bit/Byte format transfer
 - 7-bit and 10-bit device addressing modes
 - General call
 - START byte
 - Multi-master transmitter/ slave receiver mode
 - Multi-master receiver/ slave transmitter mode
 - Combined master transmit/receive and receive/transmit mode
 - Transfer rates of 10 kbps up to 400 kbps (Phillips fast-mode rate)
- Free data format
- Two DMA events (transmit and receive)
- DMA event enable/disable capability
- Seven interrupts that can be used by the CPU
- Operates with VBUS frequency from 6.7 MHz up
- Operates with module frequency between 6.7 MHz to 13.3 MHz
- Module enable/disable capability
- The SDA and SCL are optionally configurable as general purpose I/O
- Slew rate control of the outputs
- Open drain control of the outputs
- Programmable pullup/pulldown capability on the inputs
- Supports Ignore NACK mode

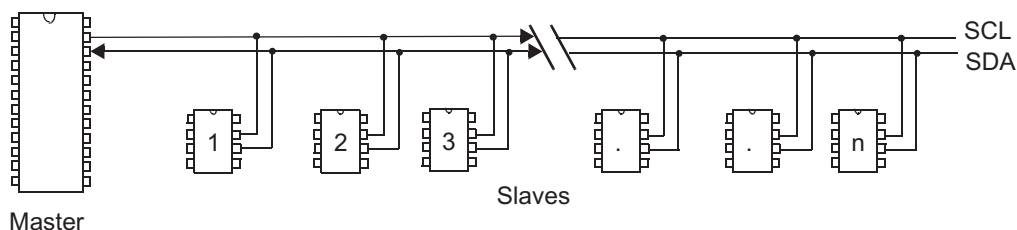
NOTE: This I2C module does **not** support:

- High-speed (HS) mode
- C-bus compatibility mode
- The combined format in 10-bit address mode (the I2C sends the slave address second byte every time it sends the slave address first byte)

30.1.1 Introduction to the I2C Module

The I2C module supports any slave or master I2C-compatible device. [Figure 30-1](#) shows an example of multiple I2C serial ports connected for a two-way transfer from one device to another device.

Figure 30-1. Multiple I2C Modules Connection Diagram



30.1.2 Functional Overview

The I2C module is a serial bus that supports multiple master devices. In multimaster mode, one or more devices can be connected to the same bus and are capable of controlling the bus. Each I2C device on the bus is recognized by a unique address and can operate as either a transmitter or a receiver, depending on the function of the device. In addition to being a transmitter or receiver, a device connected to the I2C bus can also be considered a master or a slave when performing data transfers.

NOTE: A master device is the device that initiates the data transfer on a bus and generates the clock signal that permits the transfer. During the transmission, any device addressed by the master is considered the slave.

Data is communicated to devices interfacing to the I2C module using the serial data pin (SDA) and the serial clock pin (SCL) as shown in [Figure 30-2](#). These two wires carry information between the device and the other devices connected to the I2C bus. Both SDA and SCL pins on the device are bidirectional. They must be connected to a positive supply voltage through a pull-up resistor. When the bus is free, both pins are high. The driver of these two pins has an open-drain configuration to perform the wired-AND function.

The device has a special mode that can be entered to ignore a NACK generated from non-compliant I2C devices that are incapable of generating an ACK.

The I2C module consists of the following primary blocks:

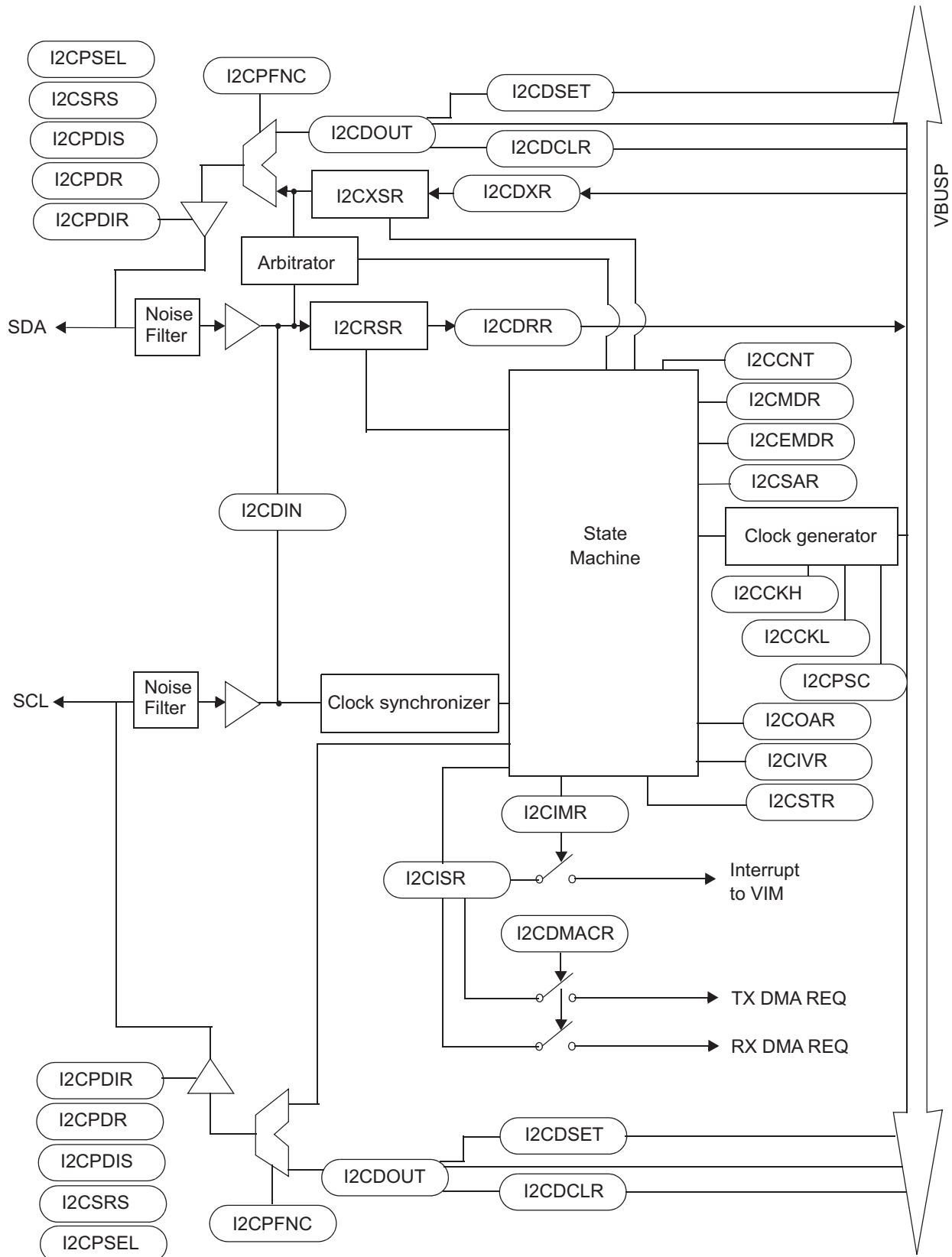
- A serial Interface: one data pin (SDA) and one clock pin (SCL)
- The device register interface
 - Data registers to temporarily hold received data and transmitted data traveling between the SDA pin and the CPU or the DMA
 - Control and status registers
- A prescaler to divide down the input clock that is driven to the I2C module
- A peripheral bus interface to enable the CPU and DMA to access the I2C module registers
- An arbitrator to handle arbitration between the I2C module (when configured as a master) and another master
- Interrupt generation logic (interrupts can be sent to the CPU)
- A clock synchronizer that synchronizes the I2C input clock (from the system module) and the clock on the SCL pin, and synchronizes data transfers with masters of different clock speeds
- A noise filter on each of the two serial pins
- DMA event generation logic that synchronizes data reception and data transmission in the I2C module for DMA transmission

In [Figure 30-2](#), the CPU or the DMA writes data for transmission to I2CDXR and reads received data from I2CDRR. When the I2C module is configured as a transmitter, data written to I2CDXR is copied to I2CXSR and shifted out one bit at a time. When the I2C module is configured as a receiver, received data is shifted into I2CRSR and then copied to I2CDRR.

When the I2C function is not needed, the pins may be controlled as general-purpose input/output (GPIO) pins. The I/O structure of each pin includes:

- programmable slew rate control of the outputs
- open drain mode
- programmable pull enable/disable on the input
- programmable pull up/pull down function on the input

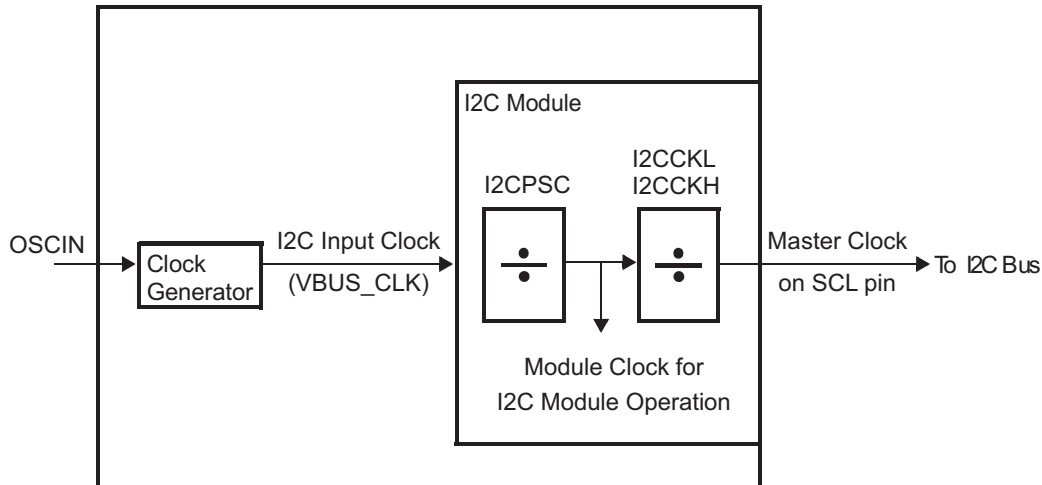
Figure 30-2. Simple I2C Block Diagram



30.1.3 Clock Generation

As shown in Figure 30-3, the I2C module uses the input clock generated from the device clock generator to generate the module clock and master clock. The I2C input clock is the device peripheral clock (VBUS_CLK). The clock is then divided twice more inside the I2C module to produce the module clock and the master clock.

Figure 30-3. Clocking Diagram for the I2C Module



The module clock determines the frequency at which the I2C module operates. A programmable prescaler in the I2C module divides down the input clock to produce the module clock. To specify the divide-down value, initialize the I2CPSC field of the prescaler register, I2CPSC. The resulting frequency is:

$$ModuleClockFrequency = \frac{I2CInputClockFrequency}{(I2CPSC + 1)} \quad (63)$$

The module clock frequency must be between 6.7MHz and 13.3MHz. The prescaler can only be initialized while the I2C module is in the reset state (IRS = 0 in I2CMDR). The prescaled frequency takes effect only when IRS is changed to 1. Changing the I2CPSC value while IRS = 1 has no effect.

The master clock appears on the SCL pin when the I2C module is configured to be a master on the I2C bus. This clock controls the timing of the communication between the I2C module and a slave. As shown in Figure 30-3, a second clock divider in the I2C module divides down the module clock to produce the master clock. The clock divider uses the I2CCKL to divide down the low portion of the module clock signal and uses the I2CCKH to divide down the high portion of the module clock signal.

The resulting frequency is:

$$MasterClockFrequency = \frac{ModuleClockFrequency}{(I2CCKL + d) + (I2CCKH + d)} \quad (64)$$

$$MasterClockFrequency = \frac{I2CInputClockFrequency}{(I2CPSC + 1)((I2CCKL + d) + (I2CCKH + d))} \quad (65)$$

where d depends on the value of I2CPSC:

I2CPSC	d
0	7
1	6
Greater than 1	5

NOTE: The master clock frequency defined above does not include rise/fall time and latency of the synchronizer inside the module. The actual transfer rate will be slower than the value calculated from the formula above. Also, due to the nature of SCL synchronization, the SCL clock period could change if SCL synchronization is taking place.

30.2 I2C Module Operation

The following section discusses how the I2C module operates.

30.2.1 Input and Output Voltage Levels

One clock pulse is generated by the master device for each data bit transferred. Because of a variety of different technology devices that can be connected to the I2C-bus, the levels of logic 0 (low) and logic 1 (high) are not fixed and depend on the associated level of V_{CCIO} . For details, see the device specific data sheet.

30.2.2 I2C Module Reset Conditions

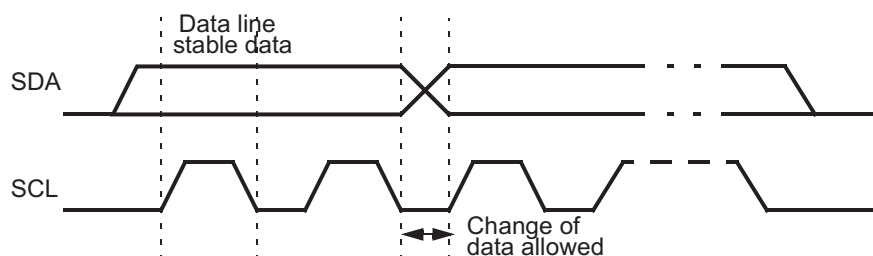
The I2C module can be reset in the following two ways:

- Through the global peripheral reset. A device reset causes a global peripheral reset.
- By clearing the $\overline{IRS}$ bit in the I2C mode register (I2CMODR). When the global peripheral reset is removed, the $\overline{IRS}$ bit is cleared to 0, keeping the I2C module in the reset state.

30.2.3 I2C Module Data Validity

The data on the SDA must be stable during the high period of the clock. See [Figure 30-4](#). The high and low state of the data line, the SDA, can only change when the clock signal is low.

Figure 30-4. Bit Transfer on the I2C Bus

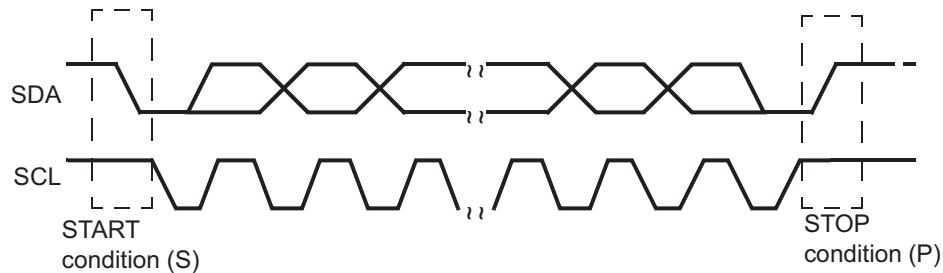


30.2.4 I2C Module Start and Stop Conditions

START and STOP conditions are generated by a master I2C module.

- The START condition is defined as a high-to-low transition on the SDA line while SCL is high. A master drives this condition to indicate the start of data transfer. The bus is considered to be busy after the START condition, and the bus busy bit (BB) in I2CSR is set to 1.
- The STOP condition is defined as a low-to-high transition on the SDA line while SCL is high. A master drives this condition to indicate the end of data transfer. The bus is considered to be free after the STOP condition, therefore the BB bit in I2CSR is cleared to 0.

Figure 30-5. I2C Module START and STOP Conditions

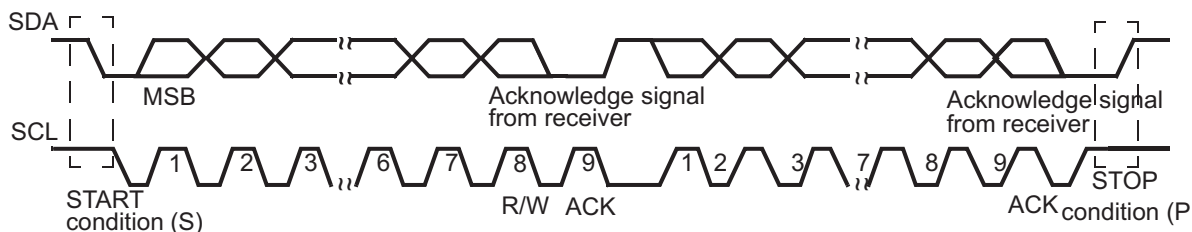


For the I2C module to start a data transfer with a START condition, the master mode bit (MST) and the START condition bit (STT) in the I2CMDR must both be set to 1. For the I2C module to end a data transfer with a STOP condition, the STOP condition bit (STP) must be set to 1. When the BB bit is set to 1 and the STT bit is set to 1, a repeated START condition is generated.

30.2.5 Serial Data Formats

The I2C module operates in byte data format. Each message put on the SDA line is 2 to 8-bits long. The number of messages that can be transmitted or received is unrestricted. The data is transferred with the most significant bit (MSB) first (Figure 30-6). Each message is followed by an acknowledge bit from the I2C if it is in receiver mode. The I2C module does not support little endian systems.

Figure 30-6. I2C Module Data Transfer



The first byte after a START condition (S) always consists of 8 bits that comprise either a 7-bit address plus the R/W bit, or 8 data bits. The eighth bit, R/W, in the first byte determines the direction of the data. When the R/W bit is 0, the master writes (transmits) data to a selected slave device; when the R/W bit is 1, the master reads (receives) data from the slave device. In acknowledge mode, an extra bit dedicated for the acknowledgement (ACK) bit is inserted after each message.

The I2C module supports the following formats:

- 7-bit addressing format (Figure 30-7)
- 10-bit addressing format (Figure 30-8)
- 7-bit/10-bit addressing format with repeated START condition (Figure 30-9)
- Free-data format (Figure 30-10)

30.2.5.1 7-Bit Addressing Format

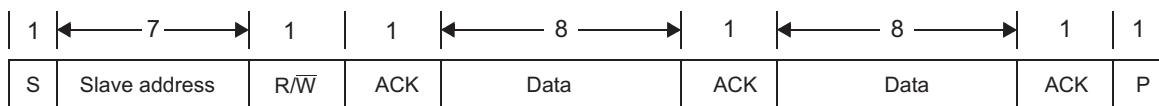
In the 7-bit addressing format (Figure 30-7), the first byte after the START condition consists of a 7-bit slave address followed by the $R/\overline{W}$ bit (in the LSB). The $R/\overline{W}$ bit determines the direction of the data transfer:

- $R/\overline{W} = 0$: The master writes (transmits) data to the addressed slave.
- $R/\overline{W} = 1$: The master reads (receives) data from the slave.

An extra clock cycle dedicated for acknowledgement (ACK) is inserted after each byte. If the ACK is inserted by the slave after the first byte from the master, it is followed by n bits of data from the transmitter (master or slave, depending on the $R/\overline{W}$ bit). The device I2C allows n to be a number between 2 to 8, programmable by the bit count (BC) field of I2CMDR. After the data bits have been transferred, the receiver inserts an ACK bit.

To select the 7-bit addressing format, write 0 to the expanded address enable (XA) bit of I2CMDR and make sure the free data format mode is off (FDF = 0 in I2CMDR).

Figure 30-7. I2C Module 7-Bit Addressing Format



30.2.5.2 10-Bit Addressing Format

The 10-bit addressing format is similar to the 7-bit addressing format, but the master sends the slave address in two separate byte transfers. In the 10-bit addressing format (Figure 30-8), the first byte is 11110b, the two MSBs of the 10-bit slave address, and the $R/\overline{W}$ bit. The ACK bit is inserted after each byte. The second byte is the remaining 8 bits of the 10-bit slave address. The slave must send an acknowledgement after each of the two byte transfers. Once the master has written the second byte to the slave, the master can either write data or use repeated a START condition to change the data direction.

To select the 10-bit addressing format, write 1 to the expanded address enable (XA) bit of I2CMDR and make sure the free data format mode is off (FDF = 0 in I2CMDR).

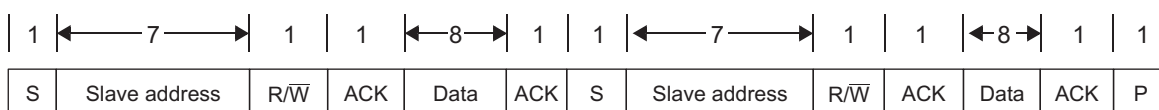
Figure 30-8. I2C Module 10-bit Addressing Format



30.2.5.3 Using the Repeated START Condition

At the end of each byte, the master can drive another START condition (Figure 30-9). Using this capability, a master can transmit/receive any number of data bytes before generating a STOP condition. The length of a data byte can be from 2 to 8 bits. The repeated START condition can be used with the 7-bit addressing, 10-bit addressing, or the free data formats.

Figure 30-9. I2C Module 7-Bit Addressing Format with Repeated START

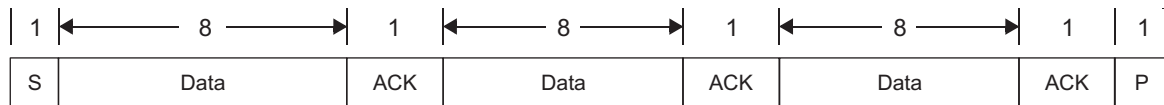


30.2.5.4 Free Data Format

In this format (Figure 30-10), the first byte after a START condition is a data byte. The ACK bit is inserted after each byte, followed by another 8 bits of data. No address or data direction bit is sent. Therefore, the transmitter and receiver must both support the free data format. The direction of data transmission (transmit or receive) remains constant throughout the transfer.

To select the free data format, write a 1 to the free data format (FDF) bit of the I2CMDR. The free data format is not supported in the digital loop back mode.

Figure 30-10. I2C Module in Free Data Format



30.2.6 NACK Bit Generation

When the I2C module is a receiver (master or slave), it can acknowledge or ignore bits sent by the transmitter. To ignore any new bits, the I2C module must send a no-acknowledge (NACK) bit during the acknowledge cycle on the bus. Table 30-1 summarizes the various ways a NACK can be generated.

Table 30-1. Ways to Generate a NACK Bit

I2C Module Condition	Basic NACK Bit Generation Options	Additional Option
Slave receiver mode	Disable data transfers (STT = 0) Allow an overrun condition (RSFULL = 1) Reset the module (IRS = 0)	Set the NACKMOD bit before the rising edge of the last data bit you intend to receive.
Master receiver mode and repeat mode (RM = 1)	Generate a STOP condition (STP = 1) Reset the module (IRS = 0)	Set the NACKMOD bit before the rising edge of the last data bit you intend to receive.
Master receiver mode with non-repeat mode (RM = 0)	If STP = 1, allow the internal data counter to count down to 0 and thus force a STOP condition. If STP = 0, make STP = 1 to generate a STOP condition. Reset the module (IRS = 0)	Set the NACKMOD bit before the rising edge of the last data bit you intend to receive.

In some applications, the slave cannot generate the ACK signal. If the IGNACK bit is set in the I2CEMDR register, the resulting NACK will be ignored and the I2C block will continue the data transfer.

30.3 I2C Operation Modes

30.3.1 Master Transmitter Mode

All masters begin in this mode. The I2C module is a master and transmits control information and data to a slave. In this mode, data assembled in any of the addressing formats shown in [Figure 30-7](#), [Figure 30-8](#), or [Figure 30-9](#) is shifted out onto the SDA pin and synchronized with the self-generated clock pulses on the SCL pin. The clock pulses are inhibited and the SCL pin is held low when the intervention of the device is required ($\overline{XSMT} = 0$) after a byte has been transmitted.

NOTE: If the I2C is configured for two simultaneous master transmissions, wait until the MST and BB have been reset before performing the second master transmission.

Failure to wait for the MST and BB to reset will prevent the start condition on the second transfer from being issued and the bus BB will not be set. Typically the end of the first transfer is handled by polling BB. However, the MST bit is not reset at the same instant as the BB bit. As a result, when the second master transmission is initiated before the resetting of the MST, the MST bit for the second transfer is reset. This prevents the I2C from recognizing itself as the master, thus failing to occupy the bus.

30.3.2 Master Receiver Mode

In this mode, the I2C module is a master and receives data from a slave. This mode can only be entered from the master transmitter mode (the I2C module must first transmit a command to the slave). In any of the addressing formats shown in [Figure 30-7](#), [Figure 30-8](#), or [Figure 30-9](#), the master receiver mode is entered after the slave address byte and the R/W bit have been transmitted (if the R/W bit is 1). Serial data bits received on the SDA pin are shifted in with the self-generated clock pulses on the SCL pin. The clock pulses are inhibited and the SCL is held low when the intervention of the device is required ($RSFULL = 1$) after a byte has been received. At the end of the transfer, the master-receiver signals the end of data to the slave-transmitter by not generating an acknowledge on the last byte that was clocked out of the slave. The slave-transmitter then releases the data line allowing the master-receiver to generate a STOP condition or a repeated START condition.

In many applications, the size of the message is in the initial bytes of the message itself. Since the size of the message is not known to the master before the transmission/reception starts, the master must use the repeat mode in order to force the stop condition when the reception is completed. The repeat mode is enabled by setting the RM bit to 1. Due to the double buffer implementation on the receive side, the master must generate the stop condition ($STP = 1$) after reading the (message size - 1)th data.

30.3.3 Slave Transmitter Mode

In this mode, the I2C module is a slave and transmits data to a master. This mode can only be entered from the slave receiver mode (The I2C module must first receive a command from the master). In any of the addressing formats shown in [Figure 30-7](#), [Figure 30-8](#), or [Figure 30-9](#), the slave transmitter mode is entered if the slave address byte is the same as its own address and the R/W bit has been transmitted (if the R/W bit is set to 1). The slave transmitter shifts the serial data out on the SDA pin with the clock pulses that are generated by the master device. The slave device does not generate the clock, but it can hold the SCL pin low when intervention of the device is required ($\overline{XSMT} = 0$) after a byte has been transmitted.

30.3.4 Slave Receiver Mode

In this mode, the I2C module is a slave and receives data from a master. All slaves begin in this mode. Serial data bits received on the SDA pin are shifted in with the clock pulses that are generated by the master device. The slave device does not generate the clock, but it can hold the SCL pin low while intervention of the device is required ($RSFULL = 1$) after a byte has been received.

30.3.5 Low Power Mode

The I2C module can be placed in low-power mode by a global low-power mode initiated by the system (by writing to the Peripheral Power-Down Set Register in the Peripheral Central Resource (PCR) module.

In effect, low-power mode shuts down all the clocks to the module. In global low-power mode, no registers are visible to the software; nothing can be written to or read from any register.

30.3.6 Free Run Mode

The I2C module can be placed in free run mode when the FREE bit (I2CMDR.14) is set to 1. This bit is primarily used on an emulator when encountering a breakpoint while debugging software. When the FREE bit is set to 0, the I2C responds differently depending on whether the SCL is high or low. If the SCL is low, the I2C stops immediately and keeps driving the SCL low whether the I2C is the master transmitter or receiver. If the SCL is high, the I2C waits until the SCL becomes a low and then stops. If the I2C is a slave, it stops when the transmission/reception completes.

30.3.7 Ignore NACK Mode

The I2C module can be placed in the ignore NACK mode by setting the IGNACK bit in the I2CEMDR register. This mode allows an I2C module that is configured as a master transmitter to ignore a NACK from a slave device that is not capable of generating a proper ACK signal.

30.4 I2C Module Integrity

The following section discusses how the I2C module maintains priorities and order among signals and commands.

30.4.1 Arbitration

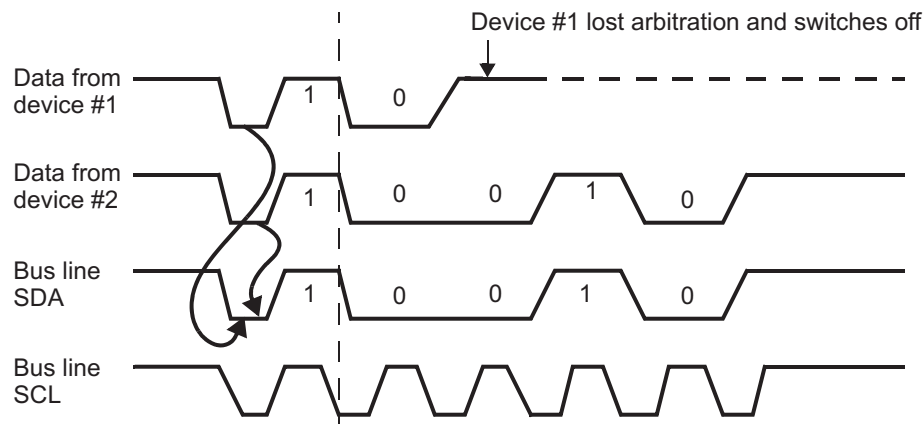
If two or more master transmitters simultaneously start a transmission on the same bus, an arbitration procedure is invoked. [Figure 30-11](#) illustrates the arbitration procedure between two devices. The arbitration procedure uses the data presented on the SDA bus by the competing transmitters. The first master transmitter that generates a high is overruled by the other master that generates a low. The arbitration procedure gives priority to the device that transmits the serial data stream with the lowest binary value. The master transmitter that loses the arbitration switches to the slave receiver mode, sets the arbitration lost (AL) flag, and generates the arbitration-lost interrupt. The data transmitted by the other master module is salvaged, and the I2C continues to receive data from the master module. Should two or more devices send identical first bytes, arbitration continues on the subsequent bytes.

If, during a serial transfer, the arbitration procedure is still in progress when a repeated START condition or STOP condition is transmitted to I2C bus, the master transmitters involved must send the repeated START condition or STOP condition at the same position in the format frame. In other words, arbitration is not allowed between:

- A repeated START condition and a data bit
- A STOP condition and a data bit
- A repeated START condition and a STOP condition

Slaves are not involved in the arbitration procedure.

Figure 30-11. Arbitration Procedure Between Two Master Transmitters



30.4.2 I2C Clock Generation and Synchronization

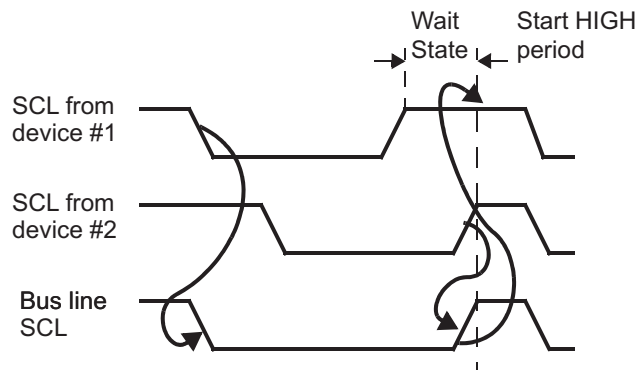
Under normal conditions only one master device generates the clock signal; the SCL. During the arbitration procedure, however, there are two or more master devices and the clock must be synchronized so that the data output can be compared. Figure 30-12 illustrates clock synchronization. The wired-AND property of the SCL line means that a device that first generates a low period on the SCL overrules the other devices. At this high-to-low transition, the clock generators of the other devices are forced to start their own low period. The SCL line is held low by the device with the longest low period. The other devices that finish their low periods must wait for the SCL line to be released before starting their high periods. A synchronized signal on the SCL is obtained where the slowest device determines the length of the low period and the fastest device determines the length of the high period.

If a device pulls down the clock line for a longer time, the result is that all clock generators must enter the wait state. In this way, a slave slows down a fast master and the slow device creates enough time to store a received byte or to prepare a byte to be transmitted.

NOTE: I2C Protocol Fault

The following conditions violate the clock spec as defined in the Philips I²C bus specification, v2.1 (*The I2C Specification*, Philips document number 9398 393 40011), and will result in an I2C protocol fault: $I2CCLKH = 2$ $I2CCLKL = 2I2CPSC = 2$. This will cause the SDA data transition to occur while the SCL is high.

Figure 30-12. Synchronization of Two I2C Clock Generators During Arbitration



30.4.3 Prescaler

The I2C module is operated by the module clock. This clock is generated by way of the I2C prescaler block. The prescaler block consists of a 8-bit register, I2CPSC, used for dividing down the device peripheral clock (VBUS_CLK) to obtain a module clock between 6.7 MHz and 13.3 MHz.

30.4.4 Noise Filter

The noise filter is used to suppress any noises that are 50ns or less. It is designed to suppress noise with one module clock, assuming the lower and upper limits of the module clock are 6.7MHz and 13.3MHz, respectively.

30.5 Operational Information

The following section provides specific information about how the I2C module operates.

30.5.1 I2C Module Interrupts

The I2C module generates seven types of interrupts. These seven interrupts are accompanied with seven interrupt mask bits in the interrupt mask register (I2CIMR) and with seven interrupt flag bits in the status register (I2CSR).

30.5.1.1 I2C Interrupt Requests

The I2C module generates the interrupt requests described below. All requests are multiplexed through an arbiter into a single I2C interrupt request to the CPU. Each interrupt request has a flag bit and an enable bit. Interrupts must be enabled prior to the occurrence of the expected interrupt condition. When one of the specified events occurs, the flag bit is set. If the corresponding enable bit is 0, the interrupt request is blocked. If the enable bit is 1, the interrupt request is forwarded to the CPU as an I2C interrupt request. As an alternative, the CPU can poll all of the bits shown in [Table 30-2](#).

Table 30-2. Interrupt Requests Generated by I2C Module

Flag	Name	Generated
AL	Arbitration-lost interrupt	Generated when the I2C module has lost an arbitration contest with another master-transmitter
NACK	No-acknowledge interrupt	Generated when the master I2C does not receive an acknowledge from the receiver
ARDY	Register-access-ready interrupt	Generated when the previously programmed address, data and command have been performed and the status bits have been updated. The interrupt is used to notify the device that the I2C registers are ready to be accessed.
RXRDY	Receive-data-ready interrupt	Generated when the received data in the receive-shift register (I2CSR) has been copied into the data receive register (I2CDRR). The RXRDY bit can also be polled by the device to determine when to read the received data in the I2CDRR.
TXRDY	Transmit-data-ready interrupt	Generated when the transmitted data has been copied from the data transmit register (I2CDXR) into the transmit-shift register (I2CXSR). The TXRDY bit can also be polled by the device to determine when to write the next data into I2CDXR.
SCD	Stop-condition-detect interrupt	Generated when a STOP condition has been detected.
AAS	Address-as-slave interrupt	Generated when the I2C has recognized its own slave address or an address of all zeroes.

The interrupt vector register (I2CIVR) contains the binary-coded-interrupt vector that indicates the highest priority interrupt that is pending and enabled. When I2CIVR is read, the corresponding interrupt flags for AL, NACK and SCD are automatically cleared, if their interrupts are enabled. Reading the I2CIVR will not clear the AAS, ARDY, RXRDY, or TXRDY interrupt pending flags. Please see [Section 30.6.3](#) for the method to clear these four flags.

If more than one interrupt is pending, a new interrupt will be generated for the next highest priority pending interrupt when you re-enable the I2C interrupt.

A transmit interrupt is generated just after the START condition in master transmitter mode. This ensures that the CPU will get an interrupt even if no slave returns an ACK to the slave address following the START condition.

It is important to note that when the I2C is configured to generate interrupts as a slave transmitter and the backward compatibility mode (BCM) bit is set to 1, an extra transmit interrupt occurs. The application should monitor the ACK from the master to determine whether to load another byte into I2CDXR.

30.5.2 DMA Controller Events

The I2C module has two events that use the DMA controller to synchronously read received data (I2CREVNT) from I2CDRR, and synchronously write data (I2CWEVNT) to the transmit buffer, I2CDXR. The read and write events have the same timing as I2CRRDY (I2CRINT) and I2CXRDY (I2CXINT), respectively.

The CPU or the DMA controller reads the received data from I2CDRR and writes the data to be transmitted to I2CDXR. The RXRDY bit is automatically cleared when the DMA controller reads the I2CDRR register, and the TXRDY bit is automatically cleared when the DMA controller writes to the I2CDXR register.

Data written to I2CDXR is copied to I2CXSR and shifted out from the SDA pin when the I2C module is configured as a transmitter. When the I2C module is configured as a receiver, received data is shifted into ICRSR and copied to I2CDRR, which can be read by the CPU or the DMA controller.

A transmit event (I2CWEVNT) is generated after a START condition in master transmitter mode. This ensures that the DMA gets an event even if no slave returns an ACK to the slave address following the START condition.

NOTE: Unexpected DMA transmit and receive event

An unexpected DMA transmit event (ICXEVT) and a DMA receive event (ICXRDY) are generated in 10-bit, master transmit, repeat mode. This event occurs soon after the start condition but before the first bit of the address is transmitted. In this event, no DMA activity should be initiated without the slave ACK being received.

30.5.3 I2C Enable/Disable

The I2C module can be enabled or disabled with the I2C reset enable bit (IRS) in the I2C module register (I2CMMDR). This occurs in one of two ways:

- Write 0 to the I2C reset bit (IRS) in I2CMMDR. All status bits are forced to the default values and the I2C mode remains disabled until IRS is changed to 1. The SDA and SCL pins are in the high impedance state.
- Initiate a device reset by driving the $\overline{\text{PORRST}}$ pin low. The entire device is reset and is held in the reset state until the pin is released and is driven high. When $\overline{\text{PORRST}}$ is released, all I2C module registers are reset to their default values. The IRS bit is forced to 0, which resets the I2C module. The I2C module stays in the reset state until a 1 is written to the IRS bit.

IRS must be 0 while the I2C module is being configured. Forcing IRS to 0 can be used to save power and also clear error conditions.

30.5.4 General Purpose I/O

Both of the I2C pins can be programmed to be general-purpose I/O pins via the I2C pin control registers (I2CPFNC, I2CDIR, I2CDOOUT, and I2CDIN).

When the I2C module is not used, the I2C pins may be programmed to be either general purpose input or general-purpose output pins. This function is controlled in the I2CDIR and I2CPFNC registers. Note that each pin can be programmed to be either an I2C pin or a GIO pin.

If the I2C function is to be used, the application software must ensure that each pin is configured as an I2C pin and not a GIO pin, or else unexpected behavior may result.

30.5.5 Pull Up/Pull Down Function

I2C module pins can have either an active pull up or active pull down that makes it possible to leave the pins unconnected externally. The pins can be programmed to have the active pull function enabled or disabled by writing to the corresponding bit in the I2CPDIS register. Please see the device-specific data sheet for the default internal pull (pull-up, pull-down or no pull) on the pins.

The pull on the pins is programmable to a setting other than the default internal pull as specified in the data sheet. The pins can be programmed to have either an active pull up or an active pull down function by writing to the corresponding bit in I2CPSEL register. The pull up/pull down function is active on the pin only when the pull enabled is programmed in the I2CPDIS register.

The pull up/pull down functions are deactivated when a bidirectional pin is configured as an output. At system reset, the pull up function of all the pins is enabled. Please see the device-specific data sheet for the current supplied by the pull up/pull down.

30.5.6 Open Drain Function

The I2C pins can be programmed to include an open drain function when they are configured as output pins. This is done by writing to the corresponding bit of the I2CPDR register. When the open drain function is enabled, a low value (0) written to the data output register forces the pin to a low output voltage (V_{OL} or lower), whereas a high value (1) written to the data output register forces the pin to a high-impedance state. The open drain function is disabled when the pin is configured as an input pin.

30.6 I2C Control Registers

[Table 30-3](#) provides a summary of the control registers. The upper word (upper 16 bits) of the registers all read as 0s. Writes have no effect on these bits. The base address for the control registers is FFF7 D400h.

Table 30-3. I2C Control Registers

Offset	Acronym	Register Description	Section
00h	I2COAR	I2C Own Address Manager	Section 30.6.1
04h	I2CIMR	I2C Interrupt Mask Register	Section 30.6.2
08h	I2CSTR	I2C Status Register	Section 30.6.3
0Ch	I2CCKL	I2C Clock Divider Low Register	Section 30.6.4
10h	I2CCKH	I2C Clock Control High Register	Section 30.6.5
14h	I2CCNT	I2C Data Count Register	Section 30.6.6
18h	I2CDRR	I2C Data Receive Register	Section 30.6.7
1Ch	I2CSAR	I2C Slave Address Register	Section 30.6.8
20h	I2CDXR	I2C Data Transmit Register	Section 30.6.9
24h	I2CMDR	I2C Mode Register	Section 30.6.10
28h	I2CIVR	I2C Interrupt Vector Register	Section 30.6.11
2Ch	I2CEMDR	I2C Extended Mode Register	Section 30.6.12
30h	I2CPSC	I2C Prescale Register	Section 30.6.13
34h	I2CPID1	I2C Peripheral ID Register 1	Section 30.6.14
38h	I2CPID2	I2C Peripheral ID Register 2	Section 30.6.15
3Ch	I2CDMACR	I2C DMA Control Register	Section 30.6.16
40h-44h	Reserved	Reserved	
48h	I2CPFNC	I2C Pin Function Register	Section 30.6.17
4Ch	I2CPDIR	I2C Pin Direction Register	Section 30.6.18
50h	I2CDIN	I2C Data Input Register	Section 30.6.19
54h	I2CDOUT	I2C Data Output Register	Section 30.6.20
58h	I2CDSET	I2C Data Set Register	Section 30.6.21
5Ch	I2CDCLR	I2C Data Clear Register	Section 30.6.22
60h	I2CPDR	I2C Pin Open Drain Register	Section 30.6.23
64h	I2CPDIS	I2C Pull Disable Register	Section 30.6.24
68h	I2CPSEL	I2C Pull Select Register	Section 30.6.25
6Ch	I2CSRS	I2C Pins Slew Rate Select Register	Section 30.6.26

30.6.1 I2C Own Address Manager (I2COAR)

The 16-bit memory-mapped I2C own address register is used to specify its own address. [Figure 30-13](#) and [Table 30-4](#) describe this register.

Figure 30-13. I2C Own Address Manager Register (I2COAR) [offset = 00h]

15	10	9	0
Reserved			OA
R-0			R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-4. I2C Own Address Manager Register (I2COAR) Field Descriptions

Bit	Field	Value	Description
15-10	Reserved	0	Reads return 0. Writes have no effect.
9-0	OA	0-3FFh	Own address These bits reflect the bus address of the I2C module. When the expand address (XA) bit I2CMDR.8 is set to 1, the I2C is in expand address mode (10-bit addressing mode). In either 7 or 10-bit address mode, all 10-bits are both readable and writable. Bits 7, 8, and 9 should only be used in 10-bit address mode. Table 30-5 provides the correct modes for these bits. Note that the user can program the I2C own address to any value as long as it does not conflict with other components in the system.

Table 30-5. Correct Mode for OA Bits

Bits Used	Mode	Value of XA
OA.6:0	7 Bit Addressing	0
OA.9:0	10 Bit Addressing	1

30.6.2 I2C Interrupt Mask Register (I2CIMR)

The 7-bit memory-mapped I2C interrupt mask register is used by the device to enable/disable the interrupts. [Figure 30-14](#) and [Table 30-6](#) describe this register.

Figure 30-14. I2C Interrupt Mask Register (I2CIMR) [offset = 04h]

15							8								
Reserved															
R-0															
7		6		5		4		3		2		1		0	
Reserved		AASEN		SCDEN		TXRDYEN		RXRDYRN		ARDYEN		NACKEN		ALEN	
R-0		RW-0		RW-0		RW-0		RW-0		RW-0		RW-0		RW-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-6. I2C Interrupt Mask Register (I2CIMR) Field Descriptions

Bit	Field	Value	Description
15-7	Reserved	0	Reads return 0. Writes have no effect.
6	AASEN	0 1	Address As Slave Interrupt Enable The AASEN interrupt is disabled. The AASEN interrupt is enabled.
5	SCDEN	0 1	Stop Condition Interrupt Enable The SCDEN interrupt is disabled. The SCDEN interrupt is enabled.
4	TXRDYEN	0 1	Transmit Data Ready Interrupt Enable The TXRDYEN interrupt is disabled. The TXRDYEN interrupt is enabled.
3	RXRDYEN	0 1	Receive Data Ready Interrupt Enable The RXRDYEN interrupt is disabled. The RXRDYEN interrupt is enabled.
2	ARDYEN	0 1	Register Access Ready Interrupt Enable The ARDYEN interrupt is disabled. The ARDYEN interrupt is enabled.
1	NACKEN	0 1	No Acknowledgement Interrupt Enable The NACKEN interrupt is disabled. The NACKEN interrupt is enabled.
0	ALEN	0 1	Arbitration Lost Interrupt Enable The ALEN interrupt is disabled. The ALEN interrupt is enabled.

30.6.3 I2C Status Register (I2CSTR)

Figure 30-15 and Table 30-7 describe this register.

Figure 30-15. I2C Status Register (I2CSR) [offset = 08h]

15	14	13	12	11	10	9	8
Reserved	SDIR	NACKSNT	BB	RSFULL	XSMT	AAS	AD0
R-0	R/W1C-0	R/W1C-0	R-0	R-0	R/W-1	R-0	R-0
7	6	5	4	3	2	1	0
Reserved	SCD	TXRDY	RXRDY	ARDY	NACK	AL	
R-0	R/W1C-0	R/W-1	R/W1C-0	R/W1C-0	R/W1C-0	R/W1C-0	R/W1C-0

LEGEND: R/W = Read/Write; R = Read only; W1C = Write 1 to clear; -n = value after reset

Table 30-7. I2C Status Register (I2CSTR) Field Descriptions

Bit	Field	Value	Description
15	Reserved	0	Reads return 0. Writes have no effect.
14	SDIR		Slave direction Setting this bit to 1 indicates that the I2C slave is a transmitter. Clearing this bit to 0 indicates that the I2C is a master transmitter/receiver or a slave receiver. This bit is also cleared by the STOP or START conditions. In DLB mode (in which the configuration should be master-transmitter slave-receiver), this bit is cleared to 0. Writing a 1 to this bit will clear it.
		0	The I2C is a master transmitter/receiver or a slave receiver.
		1	The I2C is a slave transmitter.
13	NACKSNT		No acknowledge sent This bit is set to 1 to indicate that a no acknowledgement (NACK) has been sent because the NACKMOD bit was set to 1. Writing a 1 to this bit will clear it.
		0	A NACK has not been sent.
		1	A NACK was sent because the NACKMOD was set to 1.
12	BB		Bus busy This bit indicates the state of the serial bus. On reception of a START condition or if the I2C detects a low state on I2CSCL, the device sets BB = 1. If the nIRS is set to 1 during transaction between other I2C devices, the BB bit is set at the first falling edge of SCL or START condition. BB is cleared to 0 after the reception of a STOP condition. BB is kept to 0 regardless of the SCL state when the I2C is in reset (nIRS = 0).
		0	The bus is free.
		1	The bus is busy.
11	RSFULL		Receiver shift full This bit is set to 1 to indicate that the receiver has experienced overrun. Overrun occurs when the receive shift register is full and I2CDRR has not been read since the receive shift register to I2CDRR transfer. The contents of I2CDRR are not lost. The I2C core logic is holding for I2CDRR read access. This bit is also set when, in master-repeat-mode, the I2C receives a byte of data. There is no difference between RXRDY and RSFULL in this case. The I2C master will not continue the transfer as long as the received data is in the I2CDRR or receive shift register. RSFULL is cleared when reading the I2CDRR, resetting the I2C (nIRS = 0), or resetting the device.
		0	No overrun has occurred.
		1	An overrun has occurred.

Table 30-7. I2C Status Register (I2CSTR) Field Descriptions (continued)

Bit	Field	Value	Description
10	XSMT	0 An underflow has occurred. 1 No underflow has occurred.	Transmit shift empty This bit is cleared to 0 to indicate that the transmitter has experienced underflow. Underflow occurs when the transmit shift register is empty and I2CDXR has not been loaded since the last I2CDXR to transmit shift register transfer. The I2C core logic is waiting for I2CDXR write access. XSMT is set to 1 as a result of writing to I2CDXR, by resetting the I2C block (nIRS = 0), or by resetting the device. In repeat mode, if the I2C in master transmitter mode is holding transfer with $\overline{\text{XSMT}} = 0$ (that is, waiting for further action) and the STT or STP bit is set, XSMT is set to 1 by hardware.
9	AAS	0 This bit is cleared by a STOP condition or detection of any address byte that does not match I2COAR. 1 This bit is set to 1 by the device when it has recognized its own slave address or an address of all zeros (general call).	Address as slave This bit cannot be cleared by writing a 1 to the bit or by reading the I2CIVR register.
8	AD0	0 A START or STOP condition was detected. No general call was detected. 1 An address of all zeros (general call) was detected.	Address zero status
7-6	Reserved	0	Reads return 0. Writes have no effect.
5	SCD	0 No STOP condition has been sent or received. 1 A STOP condition has been sent or received.	Stop condition detect interrupt flag This bit is set to 1 when the I2C receives or sends a STOP condition. This bit is cleared to 0 by writing a 1 to this bit or reading the value 0x0006 from I2CIVR. Writing a 1 to this bit will clear the value 0x0006 from I2CIVR.
4	TXRDY	0 I2CDXR contains data to transmit. 1 I2CDXR is empty.	Transmit data ready interrupt flag This bit is set to 1 to indicate when data in the transmit data register, I2CDXR, has been copied into the transmit shift register. This bit can also be polled by the device to indicate when to write the next transmitted data into the I2CDXR. Writing a 1 to this bit will set it. This bit is cleared to 0 and code 0x0005 in I2CIVR is cleared when the I2CDXR is written. This bit cannot be cleared by reading the I2CIVR register.
3	RXRDY	0 The I2CDRR has been read. 1 The received data has been written into the I2CDRR.	Receive data ready interrupt flag This bit is set to 1 to indicate when the data in the receive shift register has been copied into the data receive register (I2CDRR). This bit can also be polled by the device to indicate when to read the received data in the I2CDRR. Writing a 1 to this bit or reading from I2CDRR will clear this bit, and will also clear code 0x0004 from I2CIVR. This bit cannot be cleared by reading the I2CIVR register.

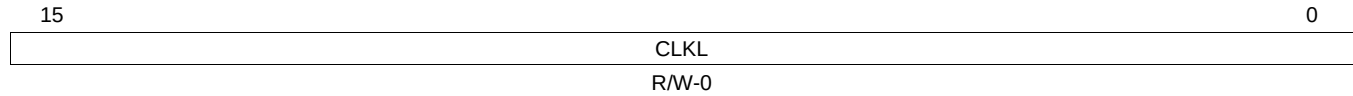
Table 30-7. I2C Status Register (I2CSTR) Field Descriptions (continued)

Bit	Field	Value	Description
2	ARDY		Register access ready interrupt flag This bit is set to 1 when the previously programmed address, data and command has been performed and the status bit has been updated. The flag is used by the device to indicate that the I2C registers are ready to be accessed again. This bit is automatically cleared by hardware when writing data to I2CDXR in transmit mode, reading data from I2CDRR in receive mode, or setting the STT or STP bit. Writing a 1 to this bit will clear this bit. This bit cannot be cleared by reading the I2CIVR register. When RM = 0, ARDY is set when I2CCNT is passed 0 if STP register bit has not been set. When RM = 1, ARDY is set at each byte end. When FDF = 0, ARDY is asserted after the ACK for the slave address. When FDF = 1, there is no slave address. Therefore, ARDY is asserted after sending the start condition.
		0	<i>Nonrepeat mode, (RM = 0):</i> I2C registers are not ready to be accessed. <i>Repeat mode (RM = 1):</i> I2C registers are not ready to be accessed.
		1	<i>Nonrepeat mode, (RM = 0):</i> ICCNT passes 0 (if STP bit has not been set). <i>Repeat mode (RM = 1):</i> The end of each byte was transmitted from I2CDXR.
1	NACK		No acknowledgement interrupt This bit is set to 1 when the master I2C does not receive an acknowledgement from the receiver. This bit is set only when the I2C has received a no-acknowledge in master mode. This bit is not set by no-acknowledgement after Start byte. In master start byte mode, the first byte (address of all zeroes) receives a NACK but does not clear the stop bit. Writing a 1 to this bit or reading the value 0x0002 from I2CIVR will clear this bit.
		0	An acknowledge was detected.
		1	No acknowledge was detected or the I2C is operating in the general call, even though an acknowledgement was received. This value clears the STP bit.
0	AL		Arbitration lost interrupt flag This bit is set to 1 when arbitration has been lost. Writing a 1 to this bit or reading the value 0x0001 from I2CIVR will clear this bit.
		0	No loss of arbitration has been detected.
		1	The device in the master transmitter mode senses it has lost an arbitration. This occurs when two or more transmitters start a transmission almost simultaneously or when the I2C attempts to start a transfer while BB=1. When this is set to 1 due to arbitration lost, the device becomes a slave receiver and the MST, STT, and STP bits in I2CMDR are cleared to 0.

30.6.4 I2C Clock Divider Low Register (I2CCKL)

The I2C clock divider low register is a 16-bit memory-mapped register used to divide the master clock down to obtain the I2C serial clock low time. [Figure 30-16](#) and [Table 30-8](#) describe this register.

Figure 30-16. I2C Clock Divider Low Register (I2CCKL) [offset = 0Ch]



LEGEND: R/W = Read/Write; -n = value after reset

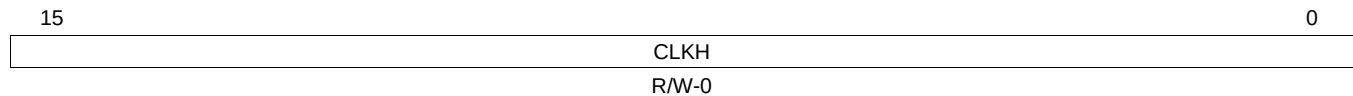
Table 30-8. I2C Clock Divider Low Register (I2CCKL) Field Descriptions

Bit	Field	Value	Description
15-0	CLKL	0-FFFFh	Low time clock division factor Used to divide down the module clock to create the low time portion of the master clock signal that will appear on the SCL pin.: $LowTime = \left(\frac{I2CCKL + d}{ModuleClockFrequency} \right) \quad (66)$ where <i>d</i> is the value that depends on the I2CPSC (see Section 30.1.3). This register must be configured while the I2C is still in reset (nIRS = 0).

30.6.5 I2C Clock Control High Register (I2CCKH)

The I2C clock divider high register is a 16-bit memory-mapped register used to divide the master clock down to obtain the I2C serial clock high time. [Figure 30-17](#) and [Table 30-9](#) describe this register.

Figure 30-17. I2C Clock Control High Register (I2CCKH) [offset = 10h]



LEGEND: R/W = Read/Write; -n = value after reset

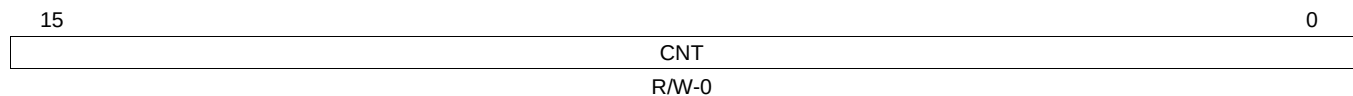
Table 30-9. I2C Clock Control High Register (I2CCKH) Field Descriptions

Bit	Field	Value	Description
15-0	CLKH	0-FFFFh	High time clock division factor Used to divide down the module clock to create the high time portion of the master clock signal that will appear on the SCL pin: $HighTime = \left(\frac{I2CCKH + d}{ModuleClockFrequency} \right) \quad (67)$ where <i>d</i> is the value that depends on the I2CPSC (see Section 30.1.3). This register must be configured while the I2C is still in reset (nIRS = 0).

30.6.6 I2C Data Count Register (I2CCNT)

The I2C data count register is a 16-bit memory-mapped register used to count received or transmitted data bytes. This register is also used to generate the STOP condition that terminates the transfer after the counter reaches zero. [Figure 30-18](#) and [Table 30-10](#) describe this register.

Figure 30-18. I2C Data Count Register (I2CCNT) [offset = 14h]



LEGEND: R/W = Read/Write; -n = value after reset

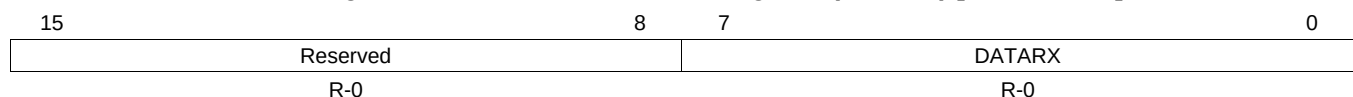
Table 30-10. I2C Data Count Register (I2CCNT) Field Descriptions

Bit	Field	Value	Description
15-0	CNT	0-FFFFh	Data counter This down counter is used to generate a stop condition, if a stop condition is specified (STP = 1). Note: I2CCNT is a don't care when RM is set to 1.
		0	The data counter is 65536.
		1	The data counter is 1.

30.6.7 I2C Data Receive Register (I2CDRR)

The I2C data receive register is a 16-bit memory-mapped register used by the device to read the received data. [Figure 30-19](#) and [Table 30-11](#) describe this register.

Figure 30-19. I2C Data Receive Register (I2CDRR) [offset = 18h]



LEGEND: R = Read only; -n = value after reset

Table 30-11. I2C Data Receive Register (I2CDRR) Field Descriptions

Bit	Field	Value	Description
15-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	DATARX	0-FFh	Receive data A read from this register clears the RXRDY bit and clears code 4h from the I2CIVR register.

30.6.8 I2C Slave Address Register (I2CSAR)

The I2C slave address register is a 16-bit memory-mapped register used to specify the address of the slave device to communicate to on the I2C bus. [Figure 30-20](#) and [Table 30-12](#) describe this register.

Figure 30-20. I2C Slave Address Register (I2CSAR) [offset = 1Ch]

15	10	9	0
Reserved			SA
R-0			R/W-3FFh

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-12. I2C Slave Address Register (I2CSAR) Field Descriptions

Bit	Field	Value	Description
15-10	Reserved	0	Reads return 0. Writes have no effect.
9-0	SA		7- or 10-bit programmable slave address In either mode, all 10-bits are readable and writable. Bits 7, 8, and 9 should only be used in 10-bit address mode. Table 30-13 lists the correct mode for each bit.

Table 30-13. Correct Mode for SA Bits

Bits Used	Mode	Value of XA
SA(6–0)	7-bit addressing	0
SA(9–0)	10-bit addressing	1

30.6.9 I2C Data Transmit Register (I2CDXR)

[Figure 30-21](#) and [Table 30-14](#) describe this register.

Figure 30-21. I2C Data Transmit Register (I2CDXR) [offset = 20h]

15	8	7	0
Reserved			DATATX
R-0			R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-14. I2C Data Transmit Register (I2CDXR) Field Descriptions

Bit	Field	Value	Description
15-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	DATATX	0-FFh	Transmit data Data written to this register will be transmitted on the I2C bus. A write to this register clears the TXRDY bit and clears code 0x05 from the I2CIVR register.

30.6.10 I2C Mode Register (I2CMDR)

Figure 30-22 and Table 30-15 describe this register.

Figure 30-22. I2C Mode Register (I2CMDR) [offset = 24h]

15	14	13	12	11	10	9	8
NACKMOD	FREE	STT	Reserved	STP	MST	TRX	XA
R/W-0	R/W-0	R/W-0	R-0	R/W-0	R/W-0	R/W-0	R/W-0
7	6	5	4	3	2		0
RM	DLB	nIRS	STB	FDF		BC	
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0		R/W-0	

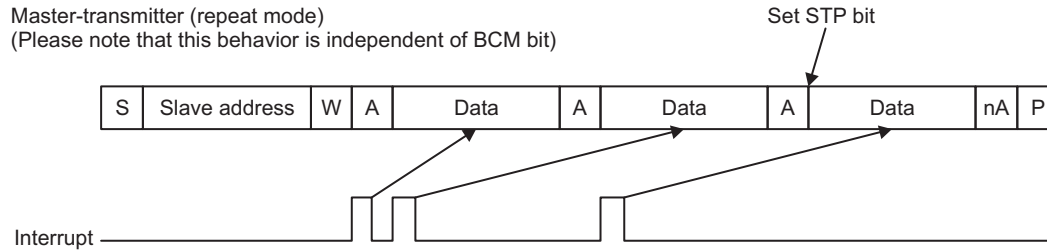
LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-15. I2C Mode Register (I2CMDR) Field Descriptions

Bit	Field	Value	Description
15	NACKMOD	0 1	No-acknowledge (NACK) mode This bit is used to send an acknowledge (ACK) or a no-acknowledge (NACK) to the transmitter. This bit is only applicable when the I2C is in receiver mode. In master receiver mode, when the internal data counter decrements to zero, the I2C sends a NACK. The master receiver I2C finishes a transfer when it sends a NACK. The I2C ignores ICCNT when NACKMOD is 1. The NACKMOD bit should be set before the rising edge of the last data bit if a NACK must be sent, and this bit is cleared once a NACK has been sent. The I2C sends an ACK to the transmitter during the acknowledge cycle. The I2C sends a NACK to the transmitter during the acknowledge cycle.
14	FREE	0 1	Free running bit This bit is used to determine the state of the I2C when a breakpoint is encountered in the high level language (HLL) debugger. The I2C stops immediately if SCL is low and keeps driving SCL low if the I2C master is a transmitter/receiver. If SCL is high, I2C waits until SCL becomes low and then stops. If the I2C is a slave, it will stop when the transmission/reception completes. The I2C runs free.
13	STT	0 1	Start condition The start condition bit works with the STP bit (master only mode). The STT and STP bits are configured to generate different transfer formats (see Table 30-16). The STT and STP bits can be used to terminate the repeat mode. This bit takes one I2C module clock cycle to set. The STT is reset to 0 by the hardware after the START condition has been generated. STT is set to 1 by the device to generate a START condition. In master mode, setting STT to 1 generates a START condition.
12	Reserved	0	Reads return 0. Writes have no effect.
11	STP	0 1	Stop condition 1(Master mode only) This bit can be set to a 1 by the CPU to generate a stop condition. It is reset to 0 by the hardware after the stop condition has been generated. The stop condition is generated when ICCNT passes 0 when the I2C is in non-repeat mode (RM=0). In repeat mode (RM=1), the stop condition is generated if STP bit is 1. In transmitter mode, I2CTXRDY needs to be 1 (that is, you have to set STP bit unless you write data into I2CDXR). STP is reset to 0 by the hardware after the STOP condition has been generated. STP is set to 1 by the device to generate a STOP condition.
10	MST	0 1	Master/slave mode bit This bit determines whether the module will operate in master or slave mode; see Table 30-17. This bit is cleared after generating a STOP condition. The BB bit is cleared first, and MST bit is cleared second. Before starting the next transaction in master mode, this bit must be confirmed to be cleared. The module is in the slave mode and the clock is received from the master device. The module is in the master mode and it generates the clock. This bit is cleared when the transfer has completed.

Table 30-15. I2C Mode Register (I2CMDR) Field Descriptions (continued)

Bit	Field	Value	Description
9	TRX	0 1	Transmit/receive bit This bit determines the direction of data transmission of the I2C module. See Table 30-17. The module is in the receive mode and data on the SDA line is shifted into the data register I2CDRR. The module is in the transmit mode and the data in the I2CDXR is shifted out on the SDA line.
8	XA	0 1	Expand address enable bit This bit controls the addressing mode. When XA is set to 1, the I2C does not support the combined format in master mode operations. However, the I2C will acknowledge and support the formats when configured as a slave. This bit needs to be configured even if the I2C is in slave mode. The mode is set to 7-bit addressing mode (normal address mode). The mode is set to 10-bit addressing mode (expanded address mode).
7	RM	0 1	Repeat mode enable bit (Master mode only) This bit is a 'don't care' if the module is configured in slave mode (MST = 0); see Table 30-16. Each time a byte of data is received, the user should decide whether or not to continue receiving more data. See Figure 30-23 for a diagram of this function. The mode is not in repeat mode. In repeat mode, data is continuously transmitted out of the ICDXR or received into the ICDRR until the STP bit is set to 1 regardless of ICCNT value. See Table 30-16 for module conditions.
6	DLB	0 1	Digital loop back enable bit This bit disables or enables the digital loopback mode of the I2C. This bit only applies in Master transmitter mode. Digital loop back mode is disabled. Digital loop back mode is enabled. In digital loop back mode, data transmitted out of the I2CDXR will be received in the I2CDRR. The address of the I2COAR is output on SDA.
5	nIRS	0 1	I2C reset enable bit When set to 0, this bit will place all status registers in this module to their default state. Resetting nIRS during a data transfer can hang the I2C bus. I2C is in reset. I2C is out of reset.
4	STB	0 1	Start byte mode enable bit (Master mode only) The Start byte mode bit is set to 1 by the CPU to configure the I2C in Start byte mode. The I2C sends "00000001" regardless of the I2CSAR value. Refer to the Philips I2C specification for more details. The module is not in START byte mode. The module is in START byte mode.
3	FDF	0 1	Free data format enable bit This bit configures the module to operate in free data format mode (see Table 30-17) in both master and slave modes. When FDF is 0, ARDY is asserted after ACK for the slave address. When FDF is 1, there is no slave address. Therefore, ARDY is asserted after sending the start condition. FDF mode is not supported in digital loop back mode. The module is not in free data format mode. The module is in free data format mode.
2-0	BC		Bit count This bit defines the number of bits starting from the LSB (excluding the acknowledge bit) that are sent on the bus when data is written to the data transmit register. If the bits BC0, BC1, and BC2 are all 0, then the number of bits sent on the bus is 8. If the bit count bits are a non-zero value, then the number of bits sent on the bus is that value. The value "001" is reserved. When performing a transfer using the bit count of, for example, n (where n is nonzero), only the n least significant bits in the data receive register are valid and correct. The rest of the bits should be disregarded. See Table 30-18 for more information.

Figure 30-23. Typical Timing Diagram of Repeat Mode

Table 30-16. I2C Module Condition, Bus Activity, and Mode

RM	STT	STP	Condition	Bus Activities ⁽¹⁾	Mode
0	0	0	Idle	None	N/A
0	0	1	Stop	P	N/A
0	1	0	(Repeat) Start	S-A-D..(n)..D	Repeat n
0	1	1	(Repeat) Start-Stop	S-A-D..(n)..D-P	Repeat n
1	0	0	Idle	none	N/A
1	0	1	Stop	P	N/A
1	1	0	(Repeat) Start	S-A-D-D-D-....	Continuous
1	1	1	Reserved	none	N/A

⁽¹⁾ P = Stop condition; S = Start condition; A = Acknowledge bit; D = data

Table 30-17. I2C Module Operating Modes

FDF	MST	TRX	Operating Mode
0	0	x	Slave in non-FDF mode
0	1	0	Master receive in non-FDF mode
0	1	1	Master transmit in non-FDF mode
1	0	0	Slave receive in FDF mode
1	0	1	Slave transmit in FDF mode
1	1	0	Master receive in FDF mode
1	1	1	Master transmit in FDF mode

Table 30-18. Number of Bits Sent on Bus

BC2	BC1	BC0	Bits in FDF	Bits with ACK
0	0	0	8	9
0	0	1	NA (reserved)	NA (reserved)
0	1	0	2	3
0	1	1	3	4
1	0	0	4	5
1	0	1	5	6
1	1	0	6	7
1	1	1	7	8

30.6.11 I2C Interrupt Vector Register (I2CIVR)

The I2C interrupt vector register is a 16-bit memory-mapped register used to indicate the occurrence of an interrupt. [Figure 30-24](#) and [Table 30-19](#) describe this register.

Figure 30-24. I2C Interrupt Vector Register (I2CIVR) [offset = 28h]

15	12	11	8	7	3	2	0
Reserved		TESTMD		Reserved		INTCODE	
R-0		R/W-0		R-0		R/WC-0	

LEGEND: R/W = Read/Write; R = Read only; C = Clear; -n = value after reset

Table 30-19. I2C Interrupt Vector Register (I2CIVR) Field Descriptions

Bit	Field	Value	Description
15-12	Reserved	0	Reads return 0. Writes have no effect.
11-8	TESTMD	0-3h	Reserved for internal testing.
7-3	Reserved	0	Reads return 0. Writes have no effect.
2-0	INTCODE	0-3h	Interrupt Code Bits This binary coded interrupt vector indicates which interrupt has occurred. If there is more than one interrupt pending, reading I2CIVR provides the vector for the highest-priority interrupt that is pending. Reading the I2CIVR will clear the corresponding flags in I2CSTR for AL, NACK and SCD as long as those interrupts are enabled. A new interrupt will be generated for each pending source. Reading I2CIVR will clear the INTCODE for AL, NACK, SCD, AAS, RXRDY and TXRDY. Reading I2CIVR will not clear the INTCODE for ARDY. The INTCODE for certain codes can also be cleared by either writing a 1 to the corresponding interrupt flag bits in I2CSTR, or by reading and writing to the receive or transmit registers. See Section 30.6.3 for more details. Users must read (clear) the I2CIVR before doing another start otherwise the I2CIVR could contain incorrect (old interrupt flag) value.

Table 30-20. Interrupt Codes for INTCODE Bits

Code	INTCODE(2-0)	Interrupt Occurred
00h	000	none
01h	001 (highest priority)	Arbitration lost (AL)
02h	010	No acknowledgement (NACK)
03h	011	Register access ready (ARDY)
04h	100	Receive data ready (RXRDY)
05h	101	Transmit data ready (TXRDY)
06h	110	Stop condition detect (SCD)
07h	111 (lowest priority)	Address as slave (AAS)

30.6.12 I2C Extended Mode Register (I2CEMDR)

The I2C extended mode register is a 16-bit memory-mapped register that contains additional mode select bits. [Figure 30-25](#) and [Table 30-21](#) describe this register.

Figure 30-25. I2C Extended Mode Register (I2CEMDR) [offset = 2Ch]

15																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
----	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-21. I2C Extended Mode Register (I2CEMDR) Field Descriptions

Bit	Field	Value	Description
15-2	Reserved	0	Reads return 0. Writes have no effect.
1	IGNACK	0	Ignore NACK mode The master transmitter will operate normally, discontinue the data transfer, and set the ARDY and NACK status bits when a NACK signal is received from the slave.
		1	The master transmitter will ignore a NACK received from the slave.
0	BCM		Backwards compatibility mode When set to 1, the I2C is compatible with previous versions of the I2C. This means the TXRDY interrupt is generated in slave-transmit mode when TXRDY is set and the I2C needs more data to transmit. This behavior causes an extra TXRDY interrupt to be generated because the I2C recognizes the end of transfer after generating an interrupt for the next byte of data. When BCM is 0, the TXRDY interrupt in slave-transmit mode is generated when XSMT = 1. In this case, the I2C generates an interrupt for the next byte after receiving the ACK from previous data. The setting of this bit only applies to slave transmit mode.
		0	The I2C is not in compatibility mode.
		1	The I2C is in compatibility mode.

30.6.13 I2C Prescale Register (I2CPSC)

The I2C prescaler register is a 16-bit memory-mapped register used for dividing down the VBUS_CLK to obtain a module clock frequency between 6.7 MHz and 13.3 MHz. [Figure 30-26](#) and [Table 30-22](#) describe this register.

Figure 30-26. I2C Prescale Register (I2CPSC) [offset = 30h]

15		8	7	0
Reserved				PSC
R-0				R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-22. I2C Prescale Register (I2CPSC) Field Descriptions

Bit	Field	Value	Description
15-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	PSC	0-FFh	Prescale 8-bit prescaler to divide down the VBUS clock to obtain the I2C module clock. This register must be initialized while the I2C is still in reset (nIRS = 0). The value takes effect on the rising edge of nIRS. See Section 30.1.3 for more information.

30.6.14 I2C Peripheral ID Register 1 (I2CPID1)

Figure 30-27 and Table 30-23 describe this register.

Figure 30-27. I2C Peripheral ID Register 1 (I2CPID1) [offset = 34h]

15	8	7	0
CLASS		REVISION	
R-1		R-46h	

LEGEND: R = Read only; -n = value after reset

Table 30-23. I2C Peripheral ID Register 1 (I2CPID1) Field Descriptions

Bit	Field	Value	Description
15-8	CLASS	0-FFh	Peripheral class These bits identify the class of peripheral.
7-0	REVISION	0-FFh	Revision level of the I2C These bits identify the revision level of the I2C.

30.6.15 I2C Peripheral ID Register 2 (I2CPID2)

Figure 30-28 and Table 30-24 describe this register.

Figure 30-28. I2C Peripheral ID Register 2 (I2CPID2) [offset = 38h]

15	8	7	0
Reserved		TYPE	
R-0		R-5h	

LEGEND: R = Read only; -n = value after reset

Table 30-24. I2C Peripheral ID Register 2 (I2CPID2) Field Descriptions

Bit	Field	Value	Description
15-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	TYPE	0-FFh	Peripheral type These bits identify the type of peripheral.

30.6.16 I2C DMA Control Register (I2CDMACR)

This register contains the transmit and receive DMA enable bits. [Figure 30-29](#) and [Table 30-25](#) describe this register.

Figure 30-29. I2C DMA Control Register (I2CDMACR) [offset = 3Ch]

15		2	1	0
Reserved			TXDMAEN	RXDMAEN
R-0			R/W-1	R/W-1

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-25. I2C DMA Control Register (I2CDMACR) Field Descriptions

Bit	Field	Value	Description
15-2	Reserved	0	Reads return 0. Writes have no effect.
1	TXDMAEN	0 1	Transmitter DMA enable This bit controls the transmit DMA event pin to the system. When this bit is a 1, the DMA transmit event is enabled and the DMA can occur. When this bit is a 0, the DMA transmit event is disabled. Writing a 1 to this bit will send a TXDMA request to the DMA module if PINFUNC is also set to 0. The transmit DMA is disabled. The transmit DMA is enabled.
0	RXDMAEN	0 1	Receive DMA enable This bit controls the receive DMA event pin to the system. When this bit is 1, the DMA receive event is enabled and the DMA can occur. When this bit is a 0, the DMA receive event is disabled. The receive DMA is disabled. The receive DMA is enabled.

30.6.17 I2C Pin Function Register (I2CPFNC)

[Figure 30-30](#) and [Table 30-26](#) describe this register.

Figure 30-30. I2C Pin Function Register (I2CPFNC) [offset = 48h]

15		1	0
Reserved			PINFUNC
R-0			R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-26. I2C Pin Function Register (I2CPFNC) Field Descriptions

Bit	Field	Value	Description
15-1	Reserved	0	Reads return 0. Writes have no effect.
0	PINFUNC	0 1	SDA and SCL pin function This bit controls whether the SDA and SCL pins function as I2C pins or as I/O pins. SDA and SCL pins function as I2C pins. SDA and SCL pins function as I/O pins.

30.6.18 I2C Pin Direction Register (I2CPDIR)

This register is used to independently configure each I2C pin, when configured as a general-purpose I/O, as either an input or output. [Figure 30-31](#) and [Table 30-27](#) describe this register.

Figure 30-31. I2C Pin Direction Register (I2CPDIR) [offset = 4Ch]

15		2	1	0
Reserved			SDADIR	SCLDIR
R-0			R/W-0	R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-27. I2C Pin Direction Register (I2CPDIR) Field Descriptions

Bit	Field	Value	Description
15-2	Reserved	0	Reads return 0. Writes have no effect.
1	SDADIR	0 1	SDA direction This bit controls the direction of the I2C SDA pin when configured as a GPIO. SDA pin functions as an input. SDA pin functions as an output.
0	SCLDIR	0 1	SCL direction This bit controls the direction of the I2C SCL pin when configured as a GPIO. SCL pin functions as an input. SCL pin functions as an output.

30.6.19 I2C Data Input Register (I2CDIN)

[Figure 30-32](#) and [Table 30-28](#) describe this register.

Figure 30-32. I2C Data Input Register (I2CDIN) [offset = 50h]

15		2	1	0
Reserved			SDAIN	SCLIN
R-0			R-x	R-x

LEGEND: R = Read only; -n = value after reset; -x = Indeterminate

Table 30-28. I2C Data Input Register (I2CDIN) Field Descriptions

Bit	Field	Value	Description
15-2	Reserved	0	Reads return 0. Writes have no effect.
1	SDAIN		Serial data in The value of this bit reflects the value on the SDA pin.
0	SCLIN		Serial clock data in The value of this bit reflects the value on the SCL pin.

30.6.20 I2C Data Output Register (I2CDOUT)

This register contains the values sent to the I2C pins. [Figure 30-33](#) and [Table 30-29](#) describe this register.

Figure 30-33. I2C Data Output Register (I2CDOUT) [offset 0x54]

15		2	1	0
Reserved			SDAOUT	SCLOUT
R-0			R/W-0	R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-29. I2C Data Output Register (I2CDOUT) Field Descriptions

Bit	Field	Value	Description
15-2	Reserved	0	Reads return 0. Writes have no effect.
1	SDAOUT	0	SDA data output This function is only active if the SDA pin is configured as an I/O pin with PINFUNC = 1. This bit contains the value sent to the SDA pin. SDA pin is driven low.
		1	SDA pin is driven high.
0	SCLOUT	0	SCL data output This function is only active if the SCL pin is configured as an I/O pin with PINFUNC = 1. This bit contains the value sent to the SCL pin. SCL pin is driven low.
		1	SCL pin is driven high.

30.6.21 I2C Data Set Register (I2CDSET)

The I2CDSET register is an alias of the I2CDOUT register. [Figure 30-34](#) and [Table 30-30](#) describe this register.

Figure 30-34. I2C Data Set Register (I2CDSET) [offset = 58h]

15		2	1	0
Reserved			SDASET	SCLSET
R-0			R/W-0	R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-30. I2C Data Set Register (I2CDSET) Field Description

Bit	Field	Value	Description
15-2	Reserved	0	Reads return 0. Writes have no effect.
1	SDASET	0	Serial data set This bit is used to set the SDA GPIO pin. Read: Reads return value of SDAOUT. Write: Writing a 0 to this bit has no effect.
		1	Read: Reads return value of SDAOUT. Write: SDAOUT is set to logic high (1).
0	SCLSET	0	Serial clock set This bit is used to set the SCL GPIO pin. Read: Reads return value of SCLOUT. Write: Writing a 0 to this bit has no effect.
		1	Read: Reads return value of SCLOUT. Write: SCLOUT is set to logic high (1).

30.6.22 I2C Data Clear Register (I2CDCLR)

The I2CDCLR register is an alias of the I2CDOUT register. [Figure 30-35](#) and [Table 30-31](#) describe this register.

Figure 30-35. I2C Data Clear Register (I2CDCLR) [offset = 5Ch]

15		2	1	0
Reserved			SDACLRL	SCLCLR
R-0			R/W-0	R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-31. I2C Data Clear Register (I2CDSET) Field Descriptions

Bit	Field	Value	Description
15-2	Reserved	0	Reads return 0. Writes have no effect.
1	SDACLRL	0	Serial data clear This bit is used to clear the SDA GPIO pin. Read: Reads return value of SDAOUT. Write: Writing a 0 to this bit has no effect.
		1	Read: Reads return value of SDAOUT. Write: SDAOUT is cleared to logic low (0).
0	SCLCLR	0	Serial clock clear This bit is used to clear the SCL GPIO pin. Read: Reads return value of SCLOUT. Write: Writing a 0 to this bit has no effect.
		1	Read: Reads return value of SCLOUT. Write: SCLOUT is cleared to logic low (0).

30.6.23 I2C Pin Open Drain Register (I2CPDR)

[Figure 30-36](#) and [Table 30-32](#) describe this register.

Figure 30-36. I2C Pin Open Drain Register (I2CPDR) [offset = 60h]

15		2	1	0
Reserved			SDAPDR	SCLPDR
R-0			R/W-0	R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-32. I2C Pin Open Drain Register (I2CPDR) Field Descriptions

Bit	Field	Value	Description
15-2	Reserved	0	Reads return 0. Writes have no effect.
1	SDAPDR	0	SDA pin open drain enable The open drain function is enabled (the output voltage is V_{OL} or lower if SDAOUT = 0 and high-impedance if SDAOUT = 1).
		1	The open drain function is disabled (output voltage is V_{OL} or lower if SDAOUT = 0; V_{OH} or higher if SDAOUT = 1).
0	SCLPDR	0	SCL pin open drain enable The open drain function is enabled (the output voltage is V_{OL} or lower if SCLOUT = 0 and high-impedance if SCLOUT = 1).
		1	The open drain function is disabled (output voltage is V_{OL} or lower if SCLOUT = 0; V_{OH} or higher if SCLOUT = 1).

30.6.24 I2C Pull Disable Register (I2CPDIS)

Values in the I2CPDIS register enable or disable the pull control capability of the pins. [Figure 30-37](#) and [Table 30-33](#) describe this register.

Figure 30-37. I2C Pull Disable Register (I2CPDIS) [offset = 64h]

15		2	1	0
Reserved			SDAPDIS	SCLPDIS
R-0			R/W-1	R/W-1

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-33. I2C Pull Disable Register (I2CPDIS) Field Descriptions

Bit	Field	Value	Description
15-2	Reserved	0	Reads return 0. Writes have no effect.
1	SDAPDIS	0	SDA pull disable The pull function is enabled.
		1	The pull function is disabled.
0	SCLPDIS	0	SCL pull disable The pull function is enabled.
		1	The pull function is disabled.

30.6.25 I2C Pull Select Register (I2CPSEL)

Values in the I2CPSEL register select the pull up or pull down functions of the corresponding pins. [Figure 30-38](#) and [Table 30-34](#) describe this register.

Figure 30-38. I2C Pull Select Register (I2CPSEL) [offset = 68h]

15		2	1	0
Reserved			SDAPSEL	SCLPSEL
R-0			R/W-0	R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-34. I2C Pull Select Register (I2CPSEL) Field Descriptions

Bit	Field	Value	Description
15-2	Reserved	0	Reads return 0. Writes have no effect.
1	SDAPSEL	0	SDA pull select The pull down function is enabled.
		1	The pull up function is enabled.
0	SCLPSEL	0	SCL pull select The pull down function is enabled.
		1	The pull up function is enabled.

30.6.25.1 Summary

The behavior of the input buffer, output buffer, and the pull control is summarized in [Table 30-35](#).

Table 30-35. Input Buffer, Output Buffer, and Pull Control Behavior as GPIO Pins

Device under Reset?	Pin Direction (DIR) ⁽¹⁾⁽²⁾	Pull Disable (PULDIS) ⁽¹⁾⁽³⁾	Pull Select (PULSEL) ⁽¹⁾⁽⁴⁾	Pull Control	Output Buffer	Input Buffer
Yes	X	X	X	Enabled	Disabled	Enabled
No	0	0	0	Pull down	Disabled	Enabled
No	0	0	1	Pull up	Disabled	Enabled
No	0	1	0	Disabled	Disabled	Enabled
No	0	1	1	Disabled	Disabled	Enabled
No	1	X	X	Disabled	Enabled	Enabled

⁽¹⁾ X = Don't care

⁽²⁾ DIR = 0 for input, = 1 for output

⁽³⁾ PULDIS = 0 for enabling pull control
= 1 for disabling pull control

⁽⁴⁾ PULSEL = 0 for pull-down functionality
= 1 for pull-up functionality

30.6.26 I2C Pins Slew Rate Select Register (I2CSRS)

This register controls the slew rate of the signal on the I2C pins. [Figure 30-39](#) and [Table 30-36](#) describe this register.

Figure 30-39. I2C Pins Slew Rate Select Register (I2CSRS) [offset = 6Ch]

15	2	1	0
Reserved		SDASRS	SCLSRS
R-0		R/W-0	R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 30-36. I2C Pins Slew Rate Select Register (I2CSRS) Field Descriptions

Bit	Field	Value	Description
15-2	Reserved	0	Reads return 0. Writes have no effect.
1	SDASRS	0	SDA slew rate select The slow buffer is selected.
		1	The normal buffer is selected.
0	SCLSRS	0	SCL slew rate select The slow buffer is selected.
		1	The normal buffer is selected.

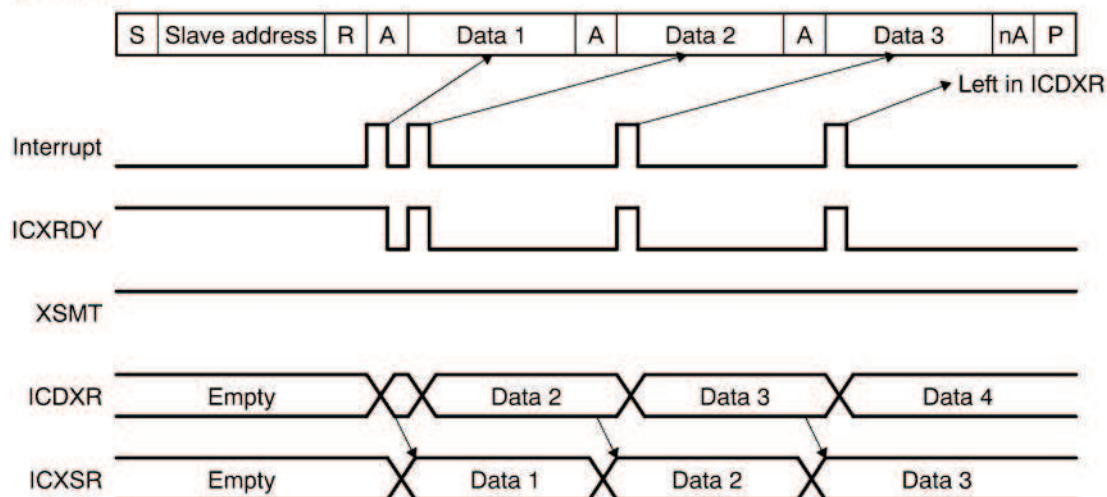
30.7 Sample Waveforms

Figure 30-40 provides waveforms to illustrate the difference between normal operation and backward compatibility mode.

Figure 30-40. Difference between Normal Operation and Backward Compatibility Mode

Slave transmitter

a) BCM=1



b) BCM=0

